

QuickFurno Claude + Codex Phase Plan

A token-saving, step-by-step workflow to redesign the existing QuickFurno website without breaking Supabase, forms, routes, or deployment.

Best decision: do not rebuild from scratch now. Modify the existing website UI/UX phase by phase, keeping the working backend, Supabase integration, lead forms, routes, environment variables, GitHub, and server deployment setup intact.

1. Strategy Summary

- Use Claude for UI/UX planning, copywriting, design system, and animation ideas.
- Use Codex in VS Code for file inspection, code implementation, bug fixing, testing, build checks, and deployment readiness.
- Use one prompt for one page or one task. Do not ask Claude or Codex to redesign the entire website in one go.
- After every phase, test locally, verify form submission, then commit changes before moving to the next phase.
- Attach the generated desktop and mobile UI images only when implementing the related page, not for every prompt.

Important QuickFurno paths

Local VS Code working folder:

C:\Users\KESHAV SHARMA\Desktop\quickfurno market\quickfurno-live-github\quickfurno

Server deployment folder:

/var/www/quickfurno-marketplace

Server login:

ssh root@213.210.21.216

Website:

https://quickfurno.in

2. Claude vs Codex Division

Task	Use Claude	Use Codex
UI/UX design system	Yes	No
Copywriting and headlines	Yes	No
Animation ideas	Yes	No
File inspection	No	Yes
Code editing	No	Yes
Supabase testing	No	Yes

Build errors	No	Yes
Deployment checks	Optional	Yes
Final launch checklist	Optional	Yes

3. Token-Saving Rules

- Never give a broad prompt like “redesign my entire website”.
- Always say exactly which page/component to change and which files/functionality not to touch.
- Do not ask Claude and Codex to solve the same task.
- Ask Claude for planning and Codex for implementation.
- Ask Codex to inspect before changing files.
- Use commits as restore points after every successful phase.

Prompt discipline example

Bad prompt:

Redesign my entire website.

Good prompt:

Only redesign the homepage. Do not touch vendor listing, vendor page, Supabase, routes, environment variables, or deployment config.

Phase 0 - Backup Before Redesign

Use	VS Code Terminal / Git
Goal	Create a safe restore point before touching the UI.
Outcome	This protects you from broken UI or accidental code changes.

Commands

```
git status
git add .
git commit -m "Backup before premium UI redesign"
```

Testing checklist:

- Confirm git status is clean or only expected files are pending.
- Do not continue until you have a backup commit.

Phase 1 - Create Final Design System

Use	Claude
Goal	Get one master UI rulebook for the whole redesign. No code yet.

Outcome	Complete this phase, test the affected area, then commit before moving to the next phase.
----------------	---

Copy-paste prompt

You are a senior UI/UX designer and frontend architect.

I am redesigning QuickFurno, a premium home-service and interior vendor marketplace in India.

Use the attached UI reference images as the final design direction.

Create a complete design system for the existing QuickFurno website without changing functionality.

Brand feel:

- Premium home-service marketplace
- Urban Company trust
- Livspace interior premium feel
- SaaS dashboard clarity
- Warm beige, ivory, charcoal, copper-gold theme
- Clean, conversion-focused, not like JustDial
- Homepage should be client-first
- Vendor cards should not appear on homepage
- Vendor listing should use compact premium cards
- Vendor page should focus on lead generation for service professionals

Output only:

1. Color palette with exact hex codes
2. Font recommendation
3. Button styles
4. Card styles
5. Section spacing
6. Animation rules
7. Mobile responsiveness rules
8. Component list for homepage, vendor listing page, vendor landing page
9. What should NOT be changed in backend/functionality

Do not write code yet.

Commands

Create a file in your project notes:

docs/ui-design-system.md

Paste Claude output there.

Testing checklist:

- Check that Claude gives exact colors, fonts, spacing, buttons, card rules, mobile rules, and component list.
- Do not spend tokens asking for implementation in this phase.

Commit after this phase

```
git add docs/ui-design-system.md
```

```
git commit -m "Add QuickFurno premium UI design system"
```

Phase 2 - Inspect Existing Code

Use	Codex in VS Code
Goal	Let Codex map files and risks before changing anything.
Outcome	This prevents accidental breaking of Supabase forms, routes, or deployment setup.

Copy-paste prompt

Inspect the existing QuickFurno project structure. Do not change any files yet. Find: 1. Homepage file/component 2. Category/vendor listing page file/component 3. Vendor page file/component 4. Lead form component 5. Supabase integration files 6. Global CSS / Tailwind / styling files 7. Routing structure 8. Any files connected to form submission Then give me a safe frontend redesign plan that keeps existing functionality, Supabase integration, environment variables, routes, and deployment setup unchanged.

Testing checklist:

- Save the important file list from Codex.
- Confirm which files contain forms and Supabase integration.
- Do not allow Codex to redesign yet.

Phase 3 - Client Homepage Redesign

Use	Codex in VS Code
Goal	Implement the premium client-first homepage only.
Outcome	Complete this phase, test the affected area, then commit before moving to the next phase.

Copy-paste prompt

Redesign only the QuickFurno client homepage based on the attached homepage desktop and mobile reference images. Important: - Do not change Supabase integration - Do not change environment variables - Do not change backend logic - Do not remove existing working form submission - Do not change deployment configuration - Do not redesign vendor listing page yet
--

- Do not redesign vendor landing page yet

Homepage requirements:

1. Premium hero section with headline:

“Tell us your requirement and get connected with verified home-service vendors near you.”

2. Add CTA buttons:

- Get Free Vendor Matches
- Explore Categories

3. No vendor cards on homepage

4. Add clean category/subcategory selector:

Parent categories:

- Interiors
- Carpentry
- Sofa
- Painting
- Civil Work

Interiors subcategories:

- Carpenters
- Modular Factory
- Premium Interiors
- Interior Designers

5. Add trust strip:

- Verified Vendors
- Fast Response
- Transparent Process
- Location-Based Matching

6. Add How QuickFurno Works section:

- Tell us your requirement
- Get matched with verified vendors
- Compare and choose
- Start your project

7. Add USP section

8. Add Pune & Mumbai coverage section

9. Add testimonials

10. Add CTA section

11. Lead enquiry form should open as popup/modal when CTA is clicked

12. Fully responsive mobile design

13. Use premium beige/charcoal/copper-gold theme

14. Use smooth, lightweight animations

15. Keep performance fast

After changes, run lint/build and tell me what changed.

Commands

```
npm run dev
```

Open locally:

```
http://localhost:3000
```

Testing checklist:

- Homepage looks close to desktop and mobile UI references.
- No vendor cards on homepage.
- CTA opens enquiry popup/modal.
- Lead form submits correctly.
- New lead appears in Supabase.
- Mobile view has no horizontal scroll.

Commit after this phase

```
git add .  
git commit -m "Redesign premium client homepage"
```

Phase 4 - Category / Vendor Listing Page

Use	Codex in VS Code
Goal	Implement compact premium vendor cards and listing experience.
Outcome	Complete this phase, test the affected area, then commit before moving to the next phase.

Copy-paste prompt

Redesign only the category/vendor listing page based on the attached vendor listing desktop and mobile reference images.

Do not change homepage now.
Do not change Supabase integration.
Do not change backend logic.
Do not change deployment config.

Vendor listing page requirements:

1. Premium compact vendor listing layout
2. Breadcrumb:
Home > Interiors > Carpenters
3. Search/filter bar:
 - City
 - Category
 - Subcategory
 - Search button
4. Quick filter chips:
 - Verified
 - Top Rated
 - Budget Friendly
 - Premium
 - Direct Factory
5. Desktop:
 - Left filter sidebar
 - Main compact vendor list
 - Right sticky lead matching card

6. Mobile:

- Top search filters
- Filter button
- Sort dropdown
- Compact stacked vendor cards
- Bottom sticky Get Matched Now CTA

7. Vendor card should include:

- Vendor logo/avatar
- Vendor name
- Verified/Premium badges
- Rating and reviews
- Location
- Service tags
- Starting price / sq ft
- Project thumbnail
- View Profile button
- Get Quote button
- Chat on WhatsApp / Ask Vendor to Contact

8. Vendor priority logic display:

- Paid/subscribed vendors appear first visually
- Non-paid vendors can still be visible
- Non-paid vendors should show subtle “Ask vendor to contact” action
- Do not hide vendors unless vendor disabled themselves

9. Keep design premium, clean, and not cluttered.

After changes, run build and tell me exactly which files changed.

Commands

```
npm run dev
npm run build
```

Testing checklist:

- Listing page desktop layout has filters, compact cards, and right lead card.
- Mobile listing has stacked compact cards and sticky CTA.
- Paid/subscribed vendors appear first visually.
- Non-paid vendors remain visible unless disabled.
- No homepage changes were made.

Commit after this phase

```
git add .
git commit -m "Redesign premium vendor listing page"
```

Phase 5 - Vendor Landing Page

Use	Codex in VS Code
Goal	Build the vendor acquisition page without affecting

	public homepage or listing page.
Outcome	Complete this phase, test the affected area, then commit before moving to the next phase.

Copy-paste prompt

Create/redesign the dedicated QuickFurno vendor landing page based on the attached vendor section design image.

Do not change client homepage.

Do not change vendor listing page.

Do not break existing routes, Supabase, or forms.

Vendor page requirements:

1. Header with For Vendors active

2. Hero headline:

“Grow your business with verified client leads from QuickFurno.”

3. CTA:

- Become a Vendor
- View Lead Demo

4. Hero dashboard preview showing:

- Active Leads
- New Requests Today
- Subscription Status
- Response Rate
- Recent Leads

5. Benefits strip:

- Verified Leads
- Faster Growth
- Transparent Lead Allocation
- Dedicated Support

6. How it works for vendors:

- Create profile
- Choose plan
- Get matched to leads
- Win more projects

7. Leads waiting for you section

8. Pricing section:

- Starter
- Growth
- Premium

9. Vendor USPs:

- Better lead quality
- City-based matching
- Priority for subscribers
- Simple dashboard
- Build trust
- No hidden charges

10. Vendor testimonials

11. Final CTA:

“Ready to receive more client enquiries?”

12. Fully responsive mobile version**13. Premium beige/charcoal/copper-gold design****14. Lightweight animation only**

Also make sure the Become a Vendor CTA opens or links to the existing vendor registration flow/form if available.

Commands

```
npm run dev
npm run build
```

Testing checklist:

- For Vendors nav is active on vendor page.
- Become a Vendor CTA connects to vendor registration flow/form.
- Pricing cards are clear.
- Dashboard preview is visually premium.
- Mobile vendor page works well.

Commit after this phase

```
git add .
git commit -m "Add premium vendor landing page"
```

Phase 6 - Forms and Supabase Safety Test

Use	Codex in VS Code
Goal	Verify every working lead/vendor form after UI redesign.
Outcome	Complete this phase, test the affected area, then commit before moving to the next phase.

Copy-paste prompt

Test all QuickFurno forms and Supabase submission flows.

Do not redesign anything.

Check:

1. Client enquiry form
2. CTA popup form
3. Vendor registration form
4. Any category enquiry form
5. Required fields validation
6. Mobile form behavior
7. Supabase insert logic
8. Error handling
9. Success message
10. Console errors

Fix only bugs related to forms or Supabase. Do not change UI design unless required for bug fix.

Commands

```
npm run dev  
npm run build
```

Testing checklist:

- Submit one client lead.
- Submit one vendor registration.
- Check Supabase tables for new entries.
- Verify required fields and success messages.
- Check browser console for errors.

Commit after this phase

```
git add .  
git commit -m "Fix and verify forms and Supabase flows"
```

Phase 7 - Mobile Polish

Use	Codex in VS Code
Goal	Make the mobile experience launch-ready across all public pages.
Outcome	Complete this phase, test the affected area, then commit before moving to the next phase.

Copy-paste prompt

Polish the mobile responsiveness of the QuickFurno website.

Do not change backend or functionality.

Check and improve:

1. Homepage mobile spacing
2. Hero mobile layout
3. Category cards mobile scrolling
4. CTA popup mobile usability
5. Vendor listing mobile cards
6. Vendor landing page mobile layout
7. Sticky CTA buttons
8. Header hamburger menu
9. Font sizes
10. Button tap sizes
11. No horizontal scroll
12. Fast loading

Use the attached mobile UI images as reference.

Commands

```
npm run dev
npm run build
```

Testing checklist:

- Homepage mobile matches reference direction.
- Vendor listing mobile cards are compact and readable.
- Vendor page mobile is not crowded.
- Sticky CTAs do not hide content.
- No horizontal scrolling.
- Buttons are easy to tap.

Commit after this phase

```
git add .
git commit -m "Polish mobile responsive UI"
```

Phase 8 - Lightweight Premium Animations

Use Claude first for the animation plan, then Codex for implementation. Keep this phase small and performance-friendly.

Claude prompt

Suggest lightweight premium animations for QuickFurno website based on the attached UI design.

Rules:

- Must not slow the website
- Must be mobile friendly
- Must feel premium, not childish
- Should improve conversion
- Should work with existing React/Next.js frontend
- Avoid heavy 3D libraries unless necessary

Give section-wise animation plan for:

1. Header
2. Hero
3. Category cards
4. CTA buttons
5. Trust strip
6. How it works section
7. Vendor cards
8. Form popup
9. Scroll reveal
10. Mobile interactions

Codex prompt

Implement lightweight premium animations across the QuickFurno frontend based on this animation plan.

Do not change functionality.

Do not add heavy libraries unless already installed.

Prefer CSS transitions, transform, opacity, subtle scroll reveal, hover lift, magnetic-like button feel, card reveal, modal animation.

Keep performance fast and mobile friendly.
Run build after implementation.

Commands

```
npm run dev  
npm run build
```

- Animations should feel premium and subtle.
- No heavy libraries unless already installed.
- No functionality should change.
- Check performance on mobile.

Commit after this phase

```
git add .  
git commit -m "Add premium lightweight animations"
```

Phase 9 - Final Production Readiness Check

Use	Codex in VS Code
Goal	Fix only launch-blocking issues and prepare final checklist.
Outcome	Complete this phase, test the affected area, then commit before moving to the next phase.

Copy-paste prompt

Perform final production readiness check for QuickFurno.

Check:

1. npm run build
2. TypeScript/JS errors
3. Console errors
4. Broken routes
5. Broken images
6. Missing alt text
7. Mobile responsiveness
8. Forms working
9. Supabase connection
10. SEO title/meta
11. Performance issues
12. Unused code if obvious
13. Deployment readiness

Do not make large redesign changes.
Fix only production-blocking issues.
Give me a final launch checklist.

Commands

```
npm run build
```

Testing checklist:

- Build passes successfully.
- No broken routes.
- No broken images.
- Forms still work.
- Supabase insert still works.
- SEO title/meta exist.
- Mobile layout is clean.

Commit after this phase

```
git add .  
git commit -m "Final production readiness fixes"
```

Phase 10 - Deploy to Server

Use	VS Code Terminal + Server SSH
Goal	Push final code to GitHub, pull on Hostinger/VPS server, build, and restart PM2.
Outcome	Deploy only after local build passes and form/Supabase tests pass.

Commands

```
# From local VS Code terminal  
git status  
git add .  
git commit -m "Final QuickFurno premium UI launch"  
git push origin main  
  
# On server  
ssh root@213.210.21.216  
cd /var/www/quickfurno-marketplace  
git pull origin main  
npm install  
npm run build  
pm2 restart all  
pm2 status
```

Testing checklist:

- Open <https://quickfurno.in>
- Check homepage desktop and mobile.
- Check listing page.
- Check vendor page.
- Submit a test lead and confirm it appears in Supabase.
- Check no console errors in browser.

4. Exact Working Order

1. Backup commit.
2. Claude: design system only.
3. Codex: inspect files only.
4. Codex: homepage redesign.
5. Test homepage and Supabase lead form.
6. Codex: vendor listing page.
7. Test listing page desktop and mobile.
8. Codex: vendor landing page.
9. Test vendor page and vendor CTA.
10. Codex: forms and Supabase safety test.
11. Codex: mobile polish.
12. Claude: animation plan.
13. Codex: implement animations.
14. Codex: final production check.
15. Deploy.

5. Which UI Images to Attach in Each Phase

Phase	Attach image	Purpose
Phase 3	Homepage desktop + homepage mobile	Client-first landing page and CTA flow
Phase 4	Vendor listing desktop + vendor listing mobile	Compact vendor cards and filters
Phase 5	Vendor section desktop image	Vendor acquisition page
Phase 7	All mobile images	Final mobile polish
Phase 8	Relevant desktop/mobile images	Animation planning only

6. Final Launch QA Checklist

- Homepage hero message is clear and conversion-focused.
- No large vendor cards appear on homepage.
- Client enquiry form opens as popup/modal and submits correctly.
- Category selector is clean and works on mobile.
- Vendor listing cards are compact and not oversized.
- Paid/subscribed vendors appear first visually.
- Non-paid vendors remain visible unless disabled.
- Vendor page explains benefits, pricing, and lead flow clearly.
- All CTAs are visible above the fold on desktop and mobile.
- All forms show success and error messages.
- New leads appear in Supabase.
- `npm run build` passes locally and on server.

- PM2 status is online after restart.
- Website loads correctly at <https://quickfurno.in>.

7. Emergency Rollback Plan

Rollback commands

```
# On server, if deployment breaks
cd /var/www/quickfurno-marketplace
git log --oneline -5

# Copy the previous safe commit hash, then run:
git reset --hard <previous-safe-commit-hash>
npm install
npm run build
pm2 restart all
pm2 status
```

Rollback only if production is broken and quick fixes are not clear. This is why every phase should be committed separately.

8. Final Recommendation

Modify the existing QuickFurno website instead of rebuilding from scratch. Treat the redesign as a full frontend refresh, but protect all working business logic. This is the fastest and safest path to launch soonest.